

CNN for MNIST Digit Classification

CYSE 635 — Module 02 Project Report

Erfan Basiri

CYSE 635

1. Introduction

Objective. I built a convolutional neural network (CNN) to classify the handwritten digits 0–9 in MNIST. Because MNIST is a relatively easy benchmark for modern CNNs, I wanted to do more than report a high accuracy. I examined the learning curves for overfitting and underfitting, looked at what the filters and feature maps respond to, tested how much the hyperparameters affect the result, and considered ways to improve accuracy or reduce computation.

Approach. My main model, *DigitNet*, is a compact residual CNN implemented in PyTorch. It has an initial 5×5 convolution followed by three residual blocks with batch normalization and squeeze-and-excitation (SE) channel attention, which gives it seven main convolutional layers. Global average pooling feeds a small classifier with two fully connected layers. During training I used mild random affine augmentation, dropout, AdamW weight decay, a ReduceLROnPlateau learning-rate schedule and early stopping. I used a separate validation set to select the checkpoint and the training settings. As a baseline, I also trained *SimpleCNN*, a simpler LeNet-style network that meets the minimum requirements of the assignment.

Key results. The reported DigitNet run correctly classified **99.70 %** of the 10,000 test images (30 errors, macro F1 0.9970), and the same setup averaged 99.63 ± 0.06 % over three seeds, compared with 99.43 % for the baseline. After the hyperparameter study, an ensemble of three models trained with a tuned setup reached **99.77 %**. The source code is on GitHub at <https://github.com/EBARV/CYSE635-MNIST-CNN> (Section 9).

2. Model Implementation

2.1 Architecture Overview

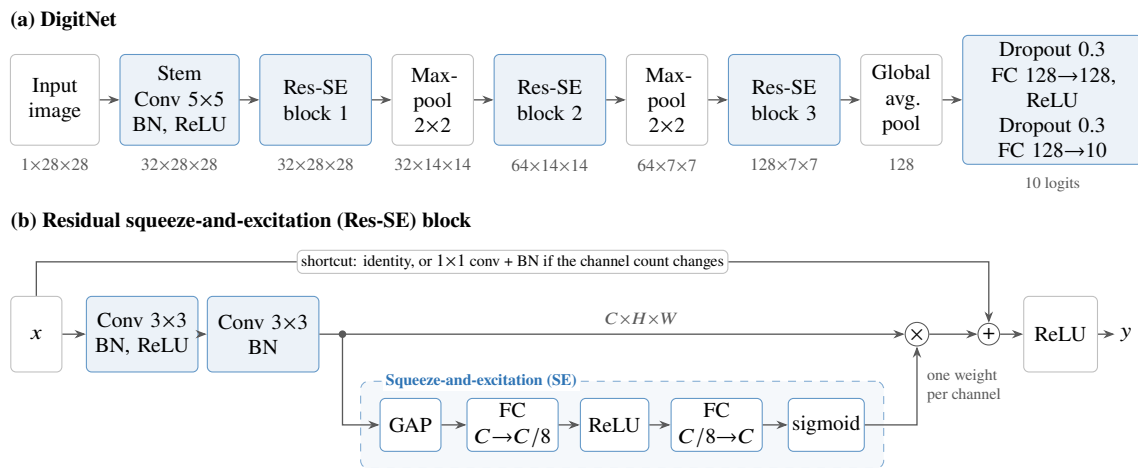


Figure 1. DigitNet architecture. (a) The full network, with the output shape (channels $\times$ height $\times$ width) under each stage. (b) One Res-SE block with C channels.

Framework. I implemented DigitNet in PyTorch 2.9.1 with CUDA 12.8 and Python 3.11.14. All code, including the model, is in the project notebook `Project-Main file.ipynb`; Figure 1 shows the full architecture and one residual SE block.

- **Convolutional layers:** seven. The stem uses 32 filters of size 5×5 . Each of the three residual blocks has two 3×3 convolutions, with 32, 64 and 128 channels. The second and third blocks also use a 1×1 projection on the shortcut to match the channel counts. Stacking 3×3 convolutions expands the receptive field without the parameter cost of making every kernel large. I chose a 5×5 stem partly because its learned filters are easier to inspect (Section 6.2).
- **Fully connected layers:** two, a $128 \rightarrow 128$ hidden layer and a $128 \rightarrow 10$ output layer. The SE modules also contain small fully connected layers (bottleneck ratio 8) that produce the channel weights.
- **Activations:** ReLU after the convolutions and the hidden classifier layer, and a sigmoid that produces the SE weights.
- **Normalization:** batch normalization follows each convolution, so those convolutions do not need biases.
- **Pooling:** two 2×2 max-pooling layers reduce the resolution from 28×28 to 14×14 and then to 7×7 , and global average pooling reduces the final 128 feature maps to 128 values. This keeps the classifier small: it has 17,802 parameters, whereas the baseline's first fully connected layer alone has roughly 400,000, and such a large layer is usually where a classic CNN overfits most.
- **Output layer:** one logit for each of the 10 digits. Cross-entropy handles the softmax during training, and I apply softmax explicitly when I need probabilities for evaluation.
- **Size and cost:** 330,726 parameters (about 1.26 MB in FP32) and about 37.6 million multiply–accumulate operations per image. That is 27 % fewer parameters than SimpleCNN, but about 3.4 times as many operations, because the early convolutions process full-resolution feature maps.

Why these design choices. *Residual shortcuts* give each block a direct path for information and gradients, so a block only has to learn a correction to its input. *Batch normalization* helps keep training stable as the activations change, which allows a higher learning rate. *Squeeze-and-excitation* lets a block adjust the relative importance of its feature channels for each input: it averages every feature map to one number, passes these numbers through a small MLP, and multiplies each channel by a weight between 0 and 1. Together, these choices let the network learn a deeper representation without making the classifier large.

Baseline. SimpleCNN follows a conventional LeNet-style pattern: Conv 5×5 (32) $\rightarrow$ ReLU $\rightarrow$ max-pool $\rightarrow$ Conv 5×5 (64) $\rightarrow$ ReLU $\rightarrow$ max-pool $\rightarrow$ dropout $\rightarrow$ FC $3136 \rightarrow 128$ $\rightarrow$ ReLU $\rightarrow$ dropout $\rightarrow$ FC $128 \rightarrow 10$. It has 454,922 parameters and needs about 11.1 million multiply–accumulate operations per image. I used it as a straightforward point of comparison for the more elaborate DigitNet.

Listing 1. Residual squeeze-and-excitation block (from the project notebook).

```

1 class ResidualSEBlock(nn.Module):
2     """Residual block: conv3x3-BN-ReLU -> conv3x3-BN -> SE -> (+ shortcut) -> ReLU."""
3
4     def __init__(self, cin, cout):
5         super().__init__()
6         self.conv1 = nn.Conv2d(cin, cout, 3, padding=1, bias=False) # no bias: BN follows
7         self.bn1 = nn.BatchNorm2d(cout)
8         self.conv2 = nn.Conv2d(cout, cout, 3, padding=1, bias=False)
9         self.bn2 = nn.BatchNorm2d(cout)
10        self.se = SqueezeExcite(cout) # channel attention
11        if cin != cout: # 1x1 projection so the shapes match for the addition
12            self.shortcut = nn.Sequential(nn.Conv2d(cin, cout, 1, bias=False), nn.BatchNorm2d(cout))
13        else:
14            self.shortcut = nn.Identity()
15
16    def forward(self, x):
17        out = F.relu(self.bn1(self.conv1(x)))
18        out = self.se(self.bn2(self.conv2(out)))
19        out = out + self.shortcut(x) # residual connection
20        return F.relu(out)

```

2.2 Regularization Techniques

I combined data augmentation, dropout, weight decay and early stopping. A learning-rate schedule controlled how training progressed, and batch normalization may also have had a small regularizing effect.

- **Dropout** ($p = 0.3$). I applied dropout before and after the hidden classifier layer, that is, on the 128 pooled features and on the 128 hidden units. Dropping units at random during training makes it harder for the classifier to rely on a small set of features. I did not add dropout inside the convolutional blocks, where batch normalization and weight sharing already limit the model. In the hyperparameter study (Section 7), changing dropout from 0 to 0.7 made little difference to accuracy, although the lowest validation losses occurred at rates of 0.3 or below.
- **Weight decay** ($\lambda = 10^{-2}$, **AdamW**). I trained with AdamW and a weight-decay coefficient of 0.01 on the convolutional and linear weights, which pulls them slightly toward zero at every step (by $\eta\lambda$), much like an L2 penalty. I excluded the biases and the BatchNorm scale and shift parameters, since shrinking them only makes it harder to re-center the activations. AdamW applies weight decay separately from its adaptive gradient update, so every weight is decayed in the same way; an ordinary L2 term added to Adam's loss would be rescaled unevenly. In this project, changing the coefficient had little effect on test accuracy.
- **Early stopping (patience 10)**. At the end of every epoch I checked the validation loss. If it failed to improve for 10 consecutive epochs, training stopped and the best validation checkpoint was restored (Listing 2). This keeps the late, overconfident epochs out of the final model and means the number of epochs does not have to be fixed in advance. A patience of 10 also leaves time for the learning-rate scheduler to respond to a plateau, since each reduction needs 3 epochs without improvement.
- **Learning-rate scheduler**. ReduceLROnPlateau monitored the validation loss and halved the learning rate (factor **0.5**) after **2** epochs without improvement, with a lower limit of 10^{-6} . This gave the optimizer relatively large steps early on and smaller steps once progress slowed. The study in Section 7 suggests that lowering the learning rate was more useful here than tuning dropout or weight decay.
- **Data augmentation** (Section 3). Small rotations, translations and changes in scale expose the model to many slightly different versions of each training digit. The aim is to make it learn the shape of a digit rather than memorize exact pixel positions.

Listing 2. Early stopping that keeps the best weights (from the project notebook).

```

1 class EarlyStopping:
2     """Stop when the validation loss has not improved for `patience` consecutive
3     epochs, and keep a copy of the best weights so they can be restored."""
4     def __init__(self, patience: int, min_delta: float = 0.0):
5         self.patience, self.min_delta = patience, min_delta
6         self.best_loss, self.best_epoch, self.counter, self.best_state = math.inf, 0, 0, None
7
8     def step(self, val_loss: float, model: nn.Module, epoch: int) -> bool:
9         if val_loss < self.best_loss - self.min_delta:      # improvement -> remember these weights
10             self.best_loss, self.best_epoch, self.counter = val_loss, epoch, 0
11             self.best_state = {k: v.detach().clone() for k, v in model.state_dict().items()}
12         else:                                              # no improvement -> count towards patience
13             self.counter += 1
14         return self.patience > 0 and self.counter >= self.patience

```

3. Data & Preprocessing

- **Dataset and split**. I loaded MNIST with `torchvision.datasets.MNIST`. It provides 60,000 labeled training images and 10,000 test images, each a 28×28 grayscale digit. I split the original training set into **55,000 images for training** and **5,000 for validation**, stratified by digit with seed 42, and kept the official **10,000 test images** separate. All three splits are close to class-balanced (each digit makes up about 9–11 %). Validation results guided training and tuning; the test results describe the selected models.
- **Normalization**. I scaled the raw pixel values from 0–255 to floats in $[0, 1]$, the same as `transforms.ToTensor()`.

The first layer of each model then standardizes them with the MNIST training mean of 0.1307 and standard deviation of 0.3081. Keeping this step inside the model lets the augmentation operate on ordinary $[0, 1]$ images.

- **Batch size and data loading.** The dataset is small enough to keep in GPU memory as float32 (roughly 200 MB). At the start of each epoch, a GPU-based mini-batch iterator (`iterate_minibatches`) shuffles the training examples and divides them into batches of **128**, including the final partial batch. It does the same job as `DataLoader (shuffle=True, batch_size=128)`, but avoids repeatedly copying small MNIST batches from the CPU to the GPU.
- **Data augmentation** (training batches only, Figure 2). I applied a separate random affine transform to each image in a training batch using `affine_grid` and `grid_sample`. Rotations were limited to $\pm 10^\circ$, scaling to 0.9–1.1, and translations to at most 2 pixels in either direction. These ranges are meant to preserve the label (a large rotation could turn a 6 into a 9) while varying the digit’s position, size and slant. Validation and test images were not augmented for the main evaluation.

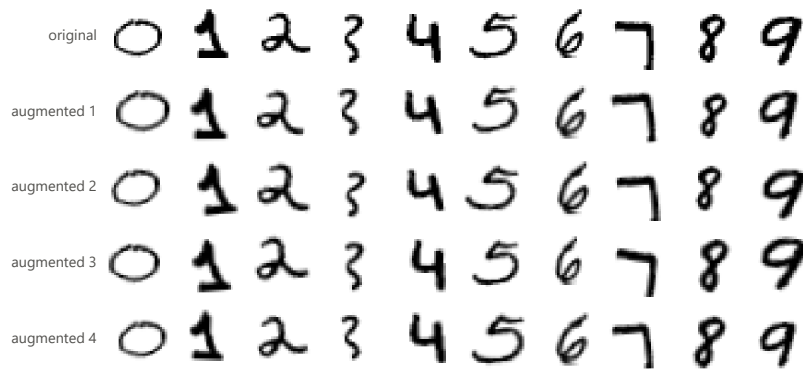


Figure 2. One training digit per class (top row) and four random augmentations of each. The augmented digits are still easy to recognize, so their labels stay correct, but their position, size and slant change. This pushes the network to learn the shape of each digit instead of exact pixel positions.

4. Training

Table 1. Training setup of the main model (the `Config()` defaults in the project notebook).

Setting	Value
Optimizer	AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.999$)
Initial learning rate	10^{-3}
Weight decay	10^{-2} on convolutional and linear weights; 0 on biases and BatchNorm parameters
Loss function	Cross-entropy (<code>F.cross_entropy</code> , which includes the softmax)
Batch size	128 (430 iterations per epoch)
LR schedule	ReduceLROnPlateau on validation loss: factor 0.5, patience 2, minimum LR 10^{-6}
Early stopping	Patience of 10 epochs on validation loss; best weights restored; at most 50 epochs
Epochs run	36 (stopped early); weights restored from epoch 26
Hardware	NVIDIA GeForce RTX 5090 (32 GB), Intel Core Ultra 9 285K, Windows 11
Training time	113 s in total (about 3.0 s per epoch, including the evaluation after each epoch)

Table 1 lists the training settings. During each epoch I shuffled the training set, augmented each mini-batch, computed the cross-entropy on the model’s logits and updated the weights with AdamW (Listing 3). After each epoch I evaluated the model without dropout or augmentation and recorded the loss on a fixed subset of 10,000 clean training images, on the validation set and on the test set. The scheduler and early stopping used only the validation loss; the test metrics were logged for the learning-curve figure and did not affect which checkpoint was kept.

Listing 3. The main part of the training loop (from `fit()` in the project notebook).

```

1 for epoch in range(1, cfg.max_epochs + 1):
2     model.train()                                # dropout on, BatchNorm uses batch statistics
3     for xb, yb in iterate_minibatches(d["x_train"], d["y_train"], cfg.batch_size):
4         if cfg.augment:
5             xb = random_augment(xb)                # random rotation / scale / shift on the GPU
6             logits = model(xb)
7             loss = F.cross_entropy(logits, yb)
8             opt.zero_grad(set_to_none=True)
9             loss.backward()                        # back-propagation
10            opt.step()                             # AdamW update (with decoupled weight decay)
11        val = evaluate(model, d["x_val"], d["y_val"]) # eval mode: no dropout, no augmentation
12        ...                                         # (clean-train and test metrics are logged too)
13        if sched is not None:
14            sched.step(val[0])                     # ReduceLROnPlateau watches validation loss
15        if stopper.step(val[0], model, epoch):      # early stopping (keeps best weights)
16            break
17    model.load_state_dict(stopper.best_state)      # restore the best-validation checkpoint

```

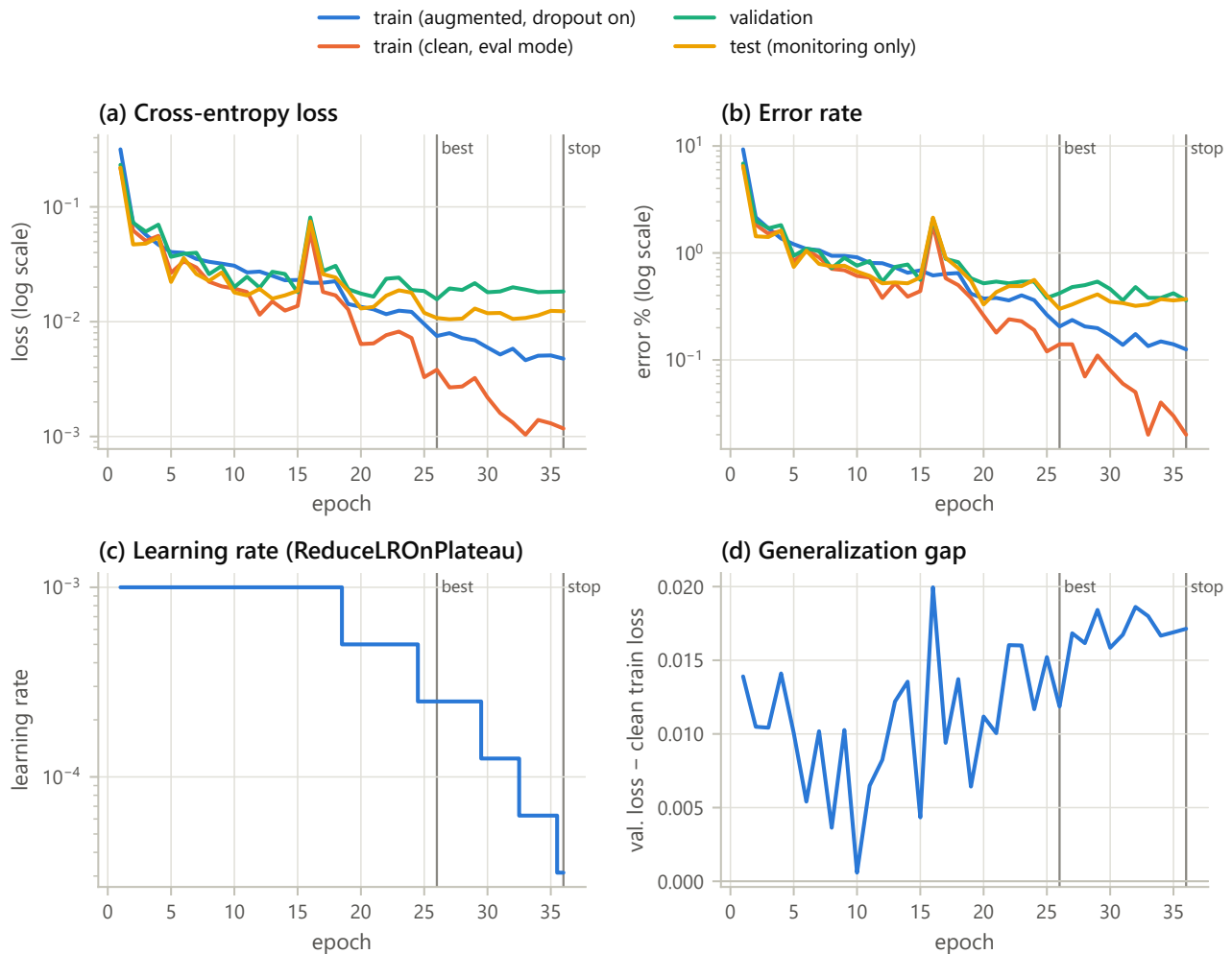


Figure 3. Learning curves of the main model. (a) Cross-entropy loss and (b) error rate after each epoch for: the augmented training batches with dropout on (blue), the same training images evaluated without augmentation or dropout (orange), the validation set (green), and the test set (yellow, only recorded for this plot). (c) The learning rate, halved by ReduceLROnPlateau. (d) The generalization gap, i.e. validation loss minus clean training loss. The vertical lines mark the best validation epoch (26), whose weights were kept, and the epoch where early stopping ended training (36). All loss and error axes use a log scale.

What happened during training (Figure 3). Validation accuracy passed 99 % by epoch 5 and then improved more slowly. A brief validation-loss spike at epoch 16 (0.08) disappeared the following epoch, which is one reason to keep the best checkpoint rather than the last one. The scheduler halved the learning rate after epochs 18, 24, 29, 32 and 35, and

the first reduction visibly pushed all loss curves down. The validation loss reached its minimum, 0.0157, at epoch 26; training stopped at epoch 36 and I restored the epoch-26 weights. The augmented training loss (blue) is higher than the clean training loss (orange) because augmentation and dropout make training harder, so I compare the clean training and validation curves when assessing overfitting.

5. Testing & Evaluation

5.1 Performance Metrics

Table 2 reports the performance of the restored seed-0 model on the 10,000 test images.

Table 2. Test results of DigitNet (seed 0). The confidence intervals come from 2,000 bootstrap resamples of the test set.

Metric	Value	Metric	Value
Accuracy	99.70 %	Cohen's κ	0.9967
95 % CI (bootstrap)	[99.59, 99.80] %	Matthews corr. coef.	0.9967
Errors (of 10,000)	30	ROC-AUC (OvR, macro)	0.99999
Macro precision	0.9970	Log-loss (NLL)	0.0108
Macro recall	0.9970	Top-2 accuracy	99.99 %
Macro F1	0.9970	Top-3 accuracy	100.00 %
Macro-F1 95 % CI	[0.9959, 0.9980]	Expected calibration error	0.0018
Weighted F1	0.9970		

- **Overall accuracy: 99.70 %** (30 errors; bootstrap 95 % CI 99.59–99.80 %). The confidence interval reflects resampling variation of this test set; it does not capture the variation caused by retraining. Seed 0 was the run I had decided to report, but it turned out to be the best of five seeds: the three-seed mean was 99.63 ± 0.06 % and the five-seed mean 99.588 % (Section 8.2). These averages say more about how the default setup performs in general. For comparison, SimpleCNN, trained the same way but without augmentation, averaged 99.43 ± 0.09 % over three seeds.
- **Macro precision 0.9970, macro recall 0.9970, macro F1 0.9970.** All three round to the same value as the accuracy, which is consistent with balanced classes and errors that are not concentrated in one digit.

5.2 Per-Class Results

Table 3. Precision, recall and F1 for each digit on the test set. “Missed” counts images of this digit that were given another label; “false alarms” counts other digits that were predicted as this one.

Digit	Precision	Recall	F1	Support	Missed (FN)	False alarms (FP)
0	0.9980	0.9990	0.9985	980	1	2
1	0.9974	0.9991	0.9982	1,135	1	3
2	0.9961	0.9971	0.9966	1,032	3	4
3	0.9970	0.9970	0.9970	1,010	3	3
4	0.9939	0.9990	0.9964	982	1	6
5	0.9966	0.9955	0.9961	892	4	3
6	0.9990	0.9948	0.9969	958	5	1
7	0.9951	0.9961	0.9956	1,028	4	5
8	0.9990	0.9969	0.9979	974	3	1
9	0.9980	0.9950	0.9965	1,009	5	2
Macro avg.	0.9970	0.9970	0.9970	10,000	30	30

- **Best classes:** digits 0, 1, 8 had the highest F1 scores (Table 3). The model missed only one zero and one one, probably because these shapes (a single closed loop, a single stroke) are the most distinctive.

- **Hardest classes:** digits 7, 5, 4 had the lowest F1 scores. A predicted 4 produced six false alarms, five of which were actually 9s. Digits 6 and 9 were missed five times each, while 7 had four misses and five false alarms, mostly mix-ups with 2.
- **Likely confusion pairs:** 4/9, 2/7 and 3/5, with six, five and five errors when both directions are counted, plus 6s predicted as 0 or 1 (two each). I look at these more closely in Section 6.3.



Figure 4. All 30 misclassified test images, from the most confident mistake to the least confident one. Each title shows “true→predicted” and how confident the model was in its wrong answer. Some are hard to read even by eye (4s with a closed top, 9s with an open loop, 7s with a curved stem), but the high-confidence mistakes also show that a confident prediction is not proof that it is right.

6. Performance Analysis

6.1 Overfitting / Underfitting

Table 4. Loss and accuracy of the restored main model on each part of the data.

Data (weights of epoch 26)	Cross-entropy	Accuracy
train (augmented, dropout on)	0.0075	99.79 %
train (clean, eval mode)	0.0038	99.86 %
validation	0.0157	99.58 %
test	0.0108	99.70 %

- **Gap between training and test loss.** At the restored checkpoint, the clean training loss was 0.0038, the validation loss 0.0157 and the test loss 0.0108 (Table 4), a cross-entropy gap of 0.0119 on validation and 0.0070 on test. In accuracy, clean training reached 99.86 %, compared with 99.58 % on validation and 99.70 % on test. The gap is small in absolute terms, but it is visible in the learning curves (Figure 3(d)).
- **When the test loss stopped improving.** The validation loss was lowest at epoch 26 and the test loss at epoch 27, only one epoch apart, so the validation set was a good stand-in for unseen data and the kept checkpoint is close to the best one in this run. After epoch 26, the clean training loss continued to fall (0.0038→0.0012) while the validation loss rose (0.0157→0.0183). That pattern suggests mild overfitting late in training: the network kept fitting the training images more confidently without improving on validation images.

- **Early stopping** triggered at epoch 36 (patience 10) and restored the epoch-26 checkpoint, so those 10 later epochs did not affect the final model.
- **Underfitting?** There is little evidence of it: the clean training accuracy reached 99.86 % and the training loss fell by orders of magnitude. The lower accuracy on augmented training batches reflects their harder inputs and active dropout, not insufficient capacity.

What overfitting looks like without regularization. To see how the regularization choices mattered, I also trained DigitNet and SimpleCNN for 50 epochs without augmentation, dropout, weight decay or early stopping (Figure 5). In both cases the training loss fell to roughly 10^{-5} – 10^{-6} while the validation loss stopped improving (its minimum came at epoch 23 for DigitNet, after a long flat stretch, and already at epoch 7 for SimpleCNN). SimpleCNN’s validation loss then rose from 0.0337 to 0.0546, the clearest sign of overfitting in this comparison. Accuracy changed less than the loss (to 99.54 % for DigitNet and 99.38 % for SimpleCNN), because a few highly confident mistakes can raise the cross-entropy a lot. The regularized DigitNet had the lowest validation loss and the highest test accuracy of the three runs.

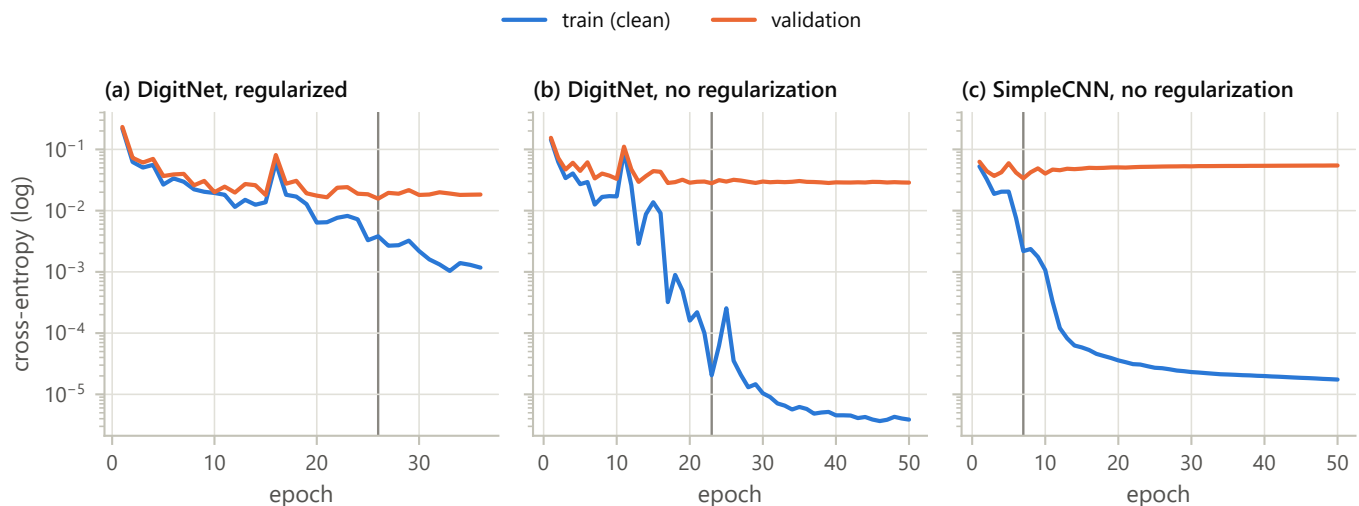


Figure 5. Clean training loss and validation loss for (a) the regularized DigitNet with early stopping, and for (b) DigitNet and (c) SimpleCNN trained for 50 epochs without any regularization. The vertical line marks the epoch with the lowest validation loss. Without regularization the training loss falls toward zero while the validation loss stays flat or goes up, i.e. the networks memorize the training set.

Conclusion: good fit. DigitNet fits the training data well and keeps most of that performance on unseen images. The remaining generalization gap is modest, and restoring the best validation checkpoint kept the later rise in validation loss out of the final model.

6.2 Filter Visualization

- **Conv layer 1 (edges, bars and blobs).** The raw 5×5 filters (Figure 6) contain bands of positive and negative weights at different orientations. Some resemble vertical, horizontal or diagonal contrast detectors (for example #26 vertical, #19 and #13 horizontal, #9 diagonal), while others look more like small blobs or center-surround patterns. They are not clean visual templates: their small size and the batch normalization that follows make the raw weights hard to interpret on their own. The averaged top patches in Figure 7(a) give a clearer view: different channels respond to short strokes in one direction, to corners and to stroke ends.
- **Conv layer 2 (curves and strokes).** A second-layer filter combines all 32 maps from the first layer, so its raw 3×3 weights are not very informative. Figure 7(b) instead shows the image patches that activate each channel most. They suggest responses to larger stroke fragments, arcs, junctions and parts of loops (as in 0, 6, 8 and 9). This is consistent with the network building more complex features from simpler early patterns, although averaged patches cannot prove that each channel has a single human-readable role.
- **Dead or uninformative filters: none found.** Every channel in every layer was active on more than 5 % of the image

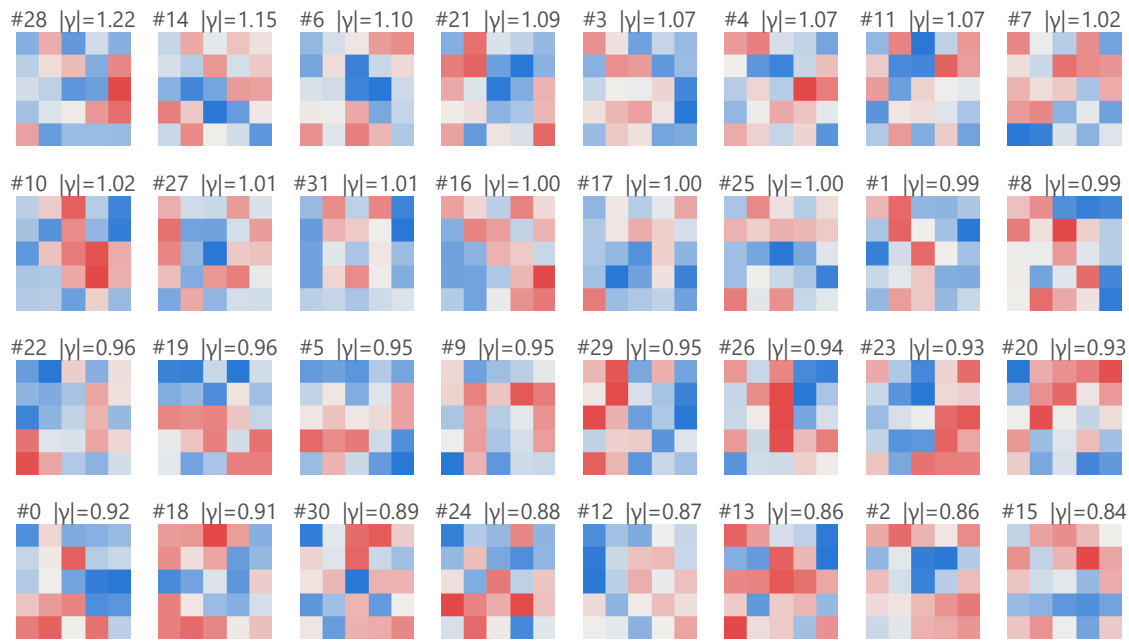


Figure 6. The 32 learned 5×5 filters of conv layer 1 (raw weights; red is positive, blue is negative, and each filter is scaled to its own largest weight). The filters are sorted by the size of their BatchNorm scale $|\gamma|$, which shows how strongly the network uses each one.

(a) conv layer 1: mean of top-25 5×5 patches



(b) conv layer 2: mean of top-25 7×7 patches



Figure 7. What each filter reacts to: for every channel, I take the 25 test images that activate it the most, cut out the patch around the strongest response in each (the size of the channel's receptive field), and average these 25 patches. (a) Conv layer 1 (5×5 receptive field); (b) conv layer 2, the first convolution in block 1 (7×7 receptive field). Dark means ink.

positions (Table 5), and the BatchNorm scales of conv layer 1 range from 0.84 to 1.22, so no filter was switched off by its scale either. The class purity rose with depth, from 0.62 on average in the stem to 0.89 in block 3, where 86 of the 128 channels have a purity of at least 0.9. Deeper features are therefore more selective for particular digits, although that does not mean each channel detects a complete digit by itself.

Feature maps (Figure 8). Following one test image of a 5 through the model, the maps after the stem still preserve much of the digit's shape, each highlighting strokes in one direction, their inside or their outline. The maps after block 1 emphasize different stroke fragments, such as the bar at the top or the loop at the bottom. As the resolution falls to 14×14 and then 7×7 , the responses become more localized and no longer look like a digit. This is what I would expect from a classifier: early layers keep local image detail, and deeper layers combine it into more abstract evidence.

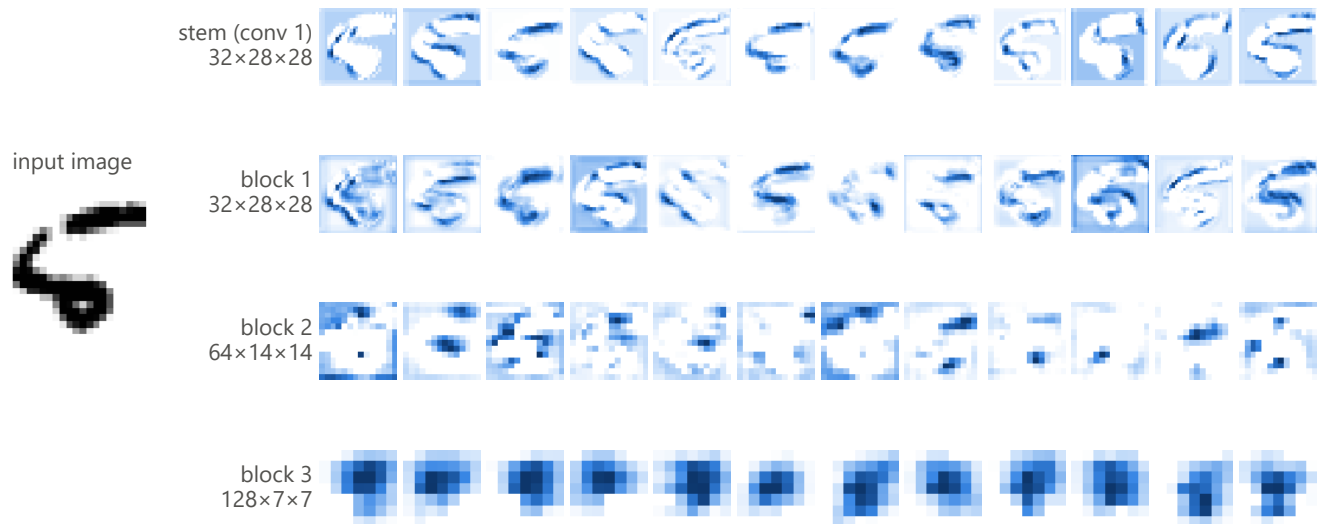


Figure 8. Feature maps of one test image (the digit 5) at each stage of DigitNet, showing the 12 most active channels per stage (darker means stronger activation). The deeper the layer, the lower the resolution and the more abstract the maps become.

Table 5. How active and how class-specific the channels of each layer are on the test set. “Purity” is the fraction of a channel’s 100 most activating test images that belong to its most common digit (0.1 means no preference, 1.0 means it only reacts to one digit).

Layer (after ReLU)	Channels	Dead (<1 % active)	Rarely active (<5 %)	Median activity	Mean class purity	Channels with purity ≥ 0.9
conv1 (stem)	32	0	0	0.66	0.62	8
conv2 (block1.conv1)	32	0	0	0.45	0.60	5
block1	32	0	0	0.70	0.57	2
block2	64	0	0	0.38	0.73	20
block3	128	0	0	0.30	0.89	86

6.3 Confusion Matrix Analysis

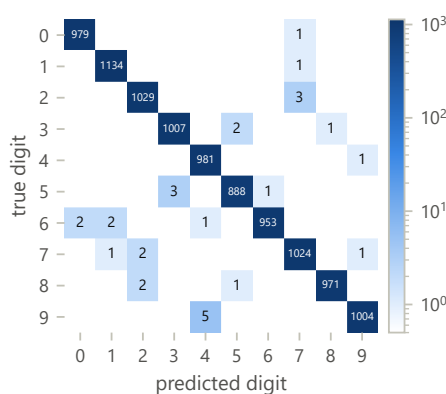


Figure 9. Test-set confusion matrix (log color scale); off-diagonal cells are the 30 errors.

- **Most common confusions** (Figure 9): a 9 predicted as 4 is the most frequent error (5 images). Next come $5 \rightarrow 3$ and $2 \rightarrow 7$ (3 each), then $7 \rightarrow 2$, $6 \rightarrow 0$, $6 \rightarrow 1$, $8 \rightarrow 2$ and $3 \rightarrow 5$ (2 each). Counting both directions, the top pairs are 4/9 (6 errors), 2/7 (5) and 3/5 (5).
- **Rows and columns with the most errors:** the rows for 6 and 9 (5 missed images each) and for 5 and 7 (4 each); the columns for 4 (6 false alarms), 7 (5) and 2 (4). The matrix is asymmetric: five 9s were labeled as 4 but only one 4 as a 9, and 6s were sometimes labeled 0 or 1, while apart from one 5 nothing was predicted as a 6.

- **Possible reasons for misclassification.** The confused digits share most of their strokes, and the mistakes often hinge on one small detail:
 - 4 vs. 9: is the top open (4) or closed (9)? A 9 written with a gap at the top, or a 4 with a closed top, is enough to flip the answer.

- 2 vs. 7: a 7 with a curved stem or a crossbar looks like a 2 without its bottom loop, and the other way around.
- 3 vs. 5: these two only differ in the direction of the top stroke.
- 6 vs. 0/1: a 6 with a very small or open loop starts to look like a 1 or an open 0.

Some images in Figure 4 are ambiguous even to me as a reader, and unusually thick, broken or strongly slanted strokes (like the broken 7 read as a 1) account for others. This helps explain the errors, although visual inspection alone cannot establish that any original label is wrong. More varied augmentation might help with some of these cases.

7. Hyperparameter Discussion

How I tested them. I varied one hyperparameter at a time while keeping the rest of the default setup fixed. Each setting was trained with seeds 0, 1 and 2, which gives 22 configurations and 66 runs. Table 6 reports the mean and standard deviation of the test accuracy, the best validation loss and the training length, and Figure 10 plots the accuracy. The default setup scored 99.70, 99.60 and 99.59 % on the three seeds, a standard deviation of about 0.06 points or roughly 6 test images, so I interpret differences smaller than about 0.1 points cautiously. I examined early-stopping patience separately, by replaying the stopping rule on complete 50-epoch runs (Figure 11).

Table 6. Hyperparameter study, changing one factor at a time (3 seeds per setting; defaults in bold). “Side effect” shows the quantity that each parameter affects the most: training loss for the learning rate, epoch time (measured alone) for the batch size, the generalization gap (validation minus clean training loss) for dropout, the total L2 norm of the weights for weight decay, and the total training time for the scheduler settings (measured while several runs shared the GPU, so only the relative values mean something).

Hyperparameter	Value	Test acc. (%)	Best val. loss	Best ep.	Epochs	Side effect
Learning rate	0.0001	99.52 ± 0.02	0.0185	26.7	36.7	train loss 0.0074
	0.0003	99.56 ± 0.07	0.0164	21.7	31.7	train loss 0.0063
	0.001	99.63 ± 0.06	0.0145	22.7	32.7	train loss 0.0040
	0.003	99.63 ± 0.06	0.0164	17.7	27.7	train loss 0.0063
	0.01	99.65 ± 0.04	0.0159	26.3	36.3	train loss 0.0043
Batch size	32	99.75 ± 0.03	0.0139	29.0	39.0	10.5 s/epoch
	64	99.64 ± 0.09	0.0147	22.0	32.0	5.3 s/epoch
	128	99.63 ± 0.06	0.0145	22.7	32.7	2.7 s/epoch
	256	99.62 ± 0.08	0.0172	24.3	34.3	1.4 s/epoch
	512	99.59 ± 0.06	0.0155	29.3	39.3	0.9 s/epoch
Dropout rate	0	99.62 ± 0.06	0.0133	21.7	31.7	gap = 0.0104
	0.15	99.64 ± 0.04	0.0147	24.0	34.0	gap = 0.0117
	0.3	99.63 ± 0.06	0.0145	22.7	32.7	gap = 0.0105
	0.5	99.60 ± 0.03	0.0190	22.3	32.3	gap = 0.0128
	0.7	99.63 ± 0.03	0.0218	24.3	34.3	gap = 0.0159
Weight decay	0	99.63 ± 0.03	0.0148	22.3	32.3	$\ W\ _2 = 38.3$
	0.0001	99.63 ± 0.01	0.0141	23.0	33.0	$\ W\ _2 = 38.0$
	0.001	99.64 ± 0.05	0.0141	24.7	34.7	$\ W\ _2 = 42.4$
	0.01	99.63 ± 0.06	0.0145	22.7	32.7	$\ W\ _2 = 42.1$
	0.1	99.64 ± 0.03	0.0143	30.0	40.0	$\ W\ _2 = 29.8$
LR factor	0.1	99.65 ± 0.04	0.0157	20.7	30.7	214 s total
	0.25	99.62 ± 0.02	0.0149	17.7	27.7	192 s total
	0.5	99.63 ± 0.06	0.0145	22.7	32.7	181 s total
	1	99.48 ± 0.05	0.0190	15.3	25.3	177 s total
LR patience	1	99.62 ± 0.06	0.0155	18.0	28.0	196 s total
	2	99.63 ± 0.06	0.0145	22.7	32.7	181 s total
	4	99.68 ± 0.05	0.0160	34.0	43.0	345 s total

- **Learning rate: convergence speed and stability.** The smallest learning rate, 10^{-4} , gave the weakest mean result

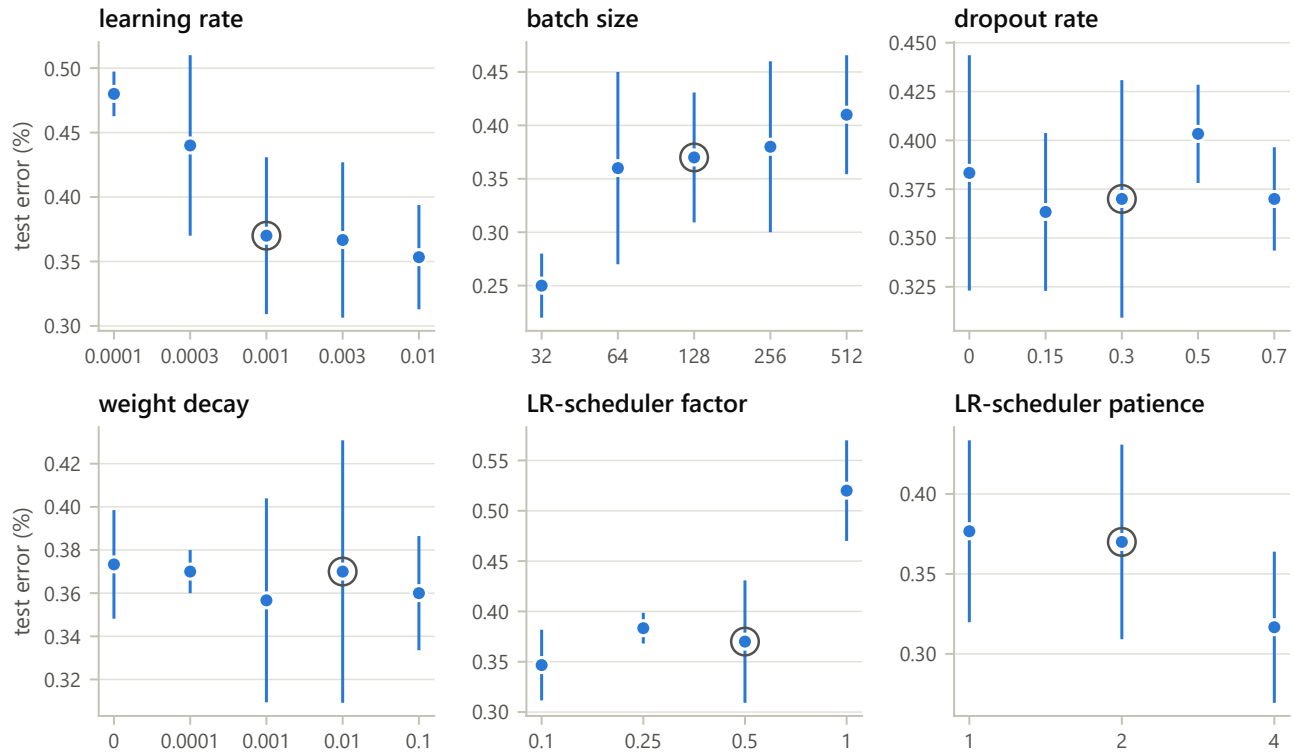


Figure 10. Test error (mean $\pm$ one standard deviation over three seeds) for every value of every hyperparameter; the circled point is the default. The vertical axes are zoomed in and differ between panels, since most differences are only a few hundredths of a percentage point.

(99.52 %), needed the most epochs (36.7 on average) and ended with the highest training loss (0.0074 vs. 0.0040), so it made too little progress within the epoch budget. Mean accuracy was much closer across 10^{-3} , 3×10^{-3} and 10^{-2} (99.63–99.65 %). 3×10^{-3} reached its best validation epoch soonest (17.7 on average), and even 10^{-2} trained without problems in these runs, helped by batch normalization, AdamW and the plateau scheduler, although that does not prove it would be equally stable for other models or data.

- Batch size: gradient noise and training time.** Batch size 32 had the highest mean test accuracy in the study, 99.75 ± 0.03 %, and the lowest best validation loss (0.0139). Mean accuracy generally fell as the batch size grew, through 99.64, 99.63 and 99.62 to 99.59 % at 512. Smaller batches give noisier gradients, which often act as a regularizer, but with the same learning rate they also make $4\times$ more updates per epoch (1,719 vs. 430), so this comparison does not isolate gradient noise as the only cause. The trade-off is runtime: the epoch time is roughly proportional to the number of steps (10.5 s for batch 32, 2.7 s for 128 and 0.87 s for 512; Table 6), because small MNIST batches cannot keep the GPU busy. Batch 128 is a practical compromise; 32 is reasonable when a small accuracy gain is worth a $\approx 4.6\times$ longer run.
- Dropout rate: regularization vs. capacity.** Test accuracy changed very little between dropout 0 and 0.7 (99.60–99.64 %, within the seed noise), but the best validation loss rose from 0.0133 at $p = 0$ to 0.0190 at $p = 0.5$ and 0.0218 at $p = 0.7$, and the generalization gap widened (Table 6). The classifier is small (17,802 parameters) and the training images are already augmented, so heavy dropout mostly takes away capacity: the classifier becomes less confident without becoming more accurate. These measurements do not show that dropout is useless in general; I would expect it to matter more for SimpleCNN, whose 400k-parameter FC layer is its main overfitting risk.
- Weight decay: weight magnitude and generalization.** Mean test accuracy stayed at 99.63–99.64 % for every coefficient from 0 to 0.1. The total weight norm stayed around 38–42 up to $\lambda = 10^{-2}$ and fell clearly only at $\lambda = 0.1$, to 29.8 (Table 6). At the default setting each step shrinks the weights by only $\eta\lambda = 10^{-5}$, which is small next to the normal updates. In this setting I found little evidence that changing weight decay improves accuracy, especially compared with the effect of the learning-rate schedule.

- **Early-stopping patience: thoroughness vs. efficiency** (Figure 11). Patience values of 1–3 tended to stop during the noisier early phase and lost 0.4–0.6 points of accuracy (99.0–99.2 %). From 5 upward the restored checkpoints were similarly accurate (99.58–99.59 %), while more patience only added epochs (22 at $p = 5$, 42 at $p = 20$). So $p \approx 5$ was the most efficient choice, and my default of 10 was conservative. Interestingly, the weights at the end of all 50 epochs scored 99.69 % on average, above the checkpoints restored by minimum validation *loss*. Validation loss can rise as a model becomes overconfident on a few examples even while its accuracy still improves. That suggests trying an accuracy-based stopping rule or a fixed schedule, chosen on validation results before looking at the test set.
- **Scheduler factor and patience: later-epoch refinement.** This was the clearest effect in the study. Disabling learning-rate reductions (factor 1.0) gave 99.48 ± 0.05 %, compared with 99.63 ± 0.06 % at the default factor of 0.5. Its clean training loss was about three times higher (0.0113), and it stopped early (25 epochs) because the validation loss kept fluctuating at the constant step size. Any factor between 0.1 and 0.5 gave similar accuracy (99.62–99.65 %). A scheduler patience of 4 gave a slightly higher mean (99.68 %) but trained much longer (43 epochs, about 1.9× the time), while a patience of 1 lowered the rate too early and stopped first (28 epochs, 99.62 %). Letting the learning rate fall mattered more than the exact decay factor.

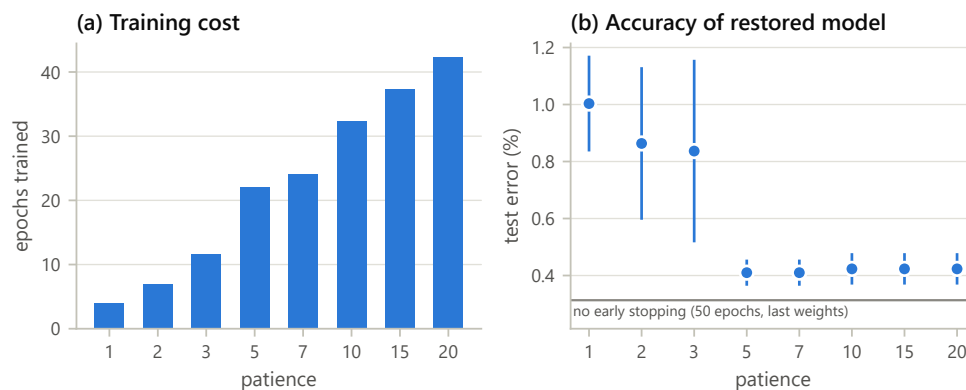


Figure 11. Early-stopping patience, replayed on three 50-epoch runs that were trained without early stopping. (a) Number of epochs trained. (b) Test error of the restored checkpoint with the best validation loss (mean $\pm$ standard deviation); the horizontal line is the error of the final weights after all 50 epochs.

What mattered most. Once training was reasonably well regularized, many settings changed the accuracy by less than the variation across seeds. The clearer effects were reducing the learning rate when the validation loss stalled (+0.15 points), avoiding a learning rate that is too small (+0.11), and using a smaller batch when runtime allowed (+0.12 at batch size 32, for a $\approx 4.6\times$ longer run). The patience study also showed that choosing a checkpoint by minimum validation loss was not always best for accuracy. Because I changed one factor at a time, the study cannot rule out interactions between settings.

8. Conclusions & Suggested Improvements

8.1 Conclusions

- **Final accuracy.** The reported DigitNet model classified 99.70 % of the MNIST test images correctly (macro F1 0.9970, Cohen's κ 0.9967, ECE 0.0018). Across three seeds the default setup averaged 99.63 ± 0.06 %, so the stronger seed-0 result should be read alongside that average. Five models with test-time augmentation reached 99.70 %, and the tuned setup averaged 99.70 ± 0.00 % (99.77 % when three models are averaged). This meets what I expected for a modern CNN on MNIST and is clearly above the SimpleCNN baseline (99.43 %), although the two differ in both architecture and augmentation, so the gap cannot be credited to the architecture alone.
- **Most effective regularization.** Together, the regularizers clearly matter: without any of them, both networks drove the training loss almost to zero while the validation loss stopped improving, and DigitNet fell to 99.54 % (Section 6.1). Among the settings I varied, the learning-rate schedule had the clearest effect (+0.15 points over a constant learning

rate), while dropout and weight decay mostly changed the validation loss and the size of the weights, not the accuracy. Augmentation and early stopping are plausible main contributors, but I did not test them in separate controlled ablations, so I treat their individual roles as explanations rather than measured effects.

- **Key insights from the analysis.**

1. The visual analysis suggests a progression from local strokes and corners in conv layer 1, to curves and junctions in conv layer 2, to channels in block 3 that are selective for particular digits (86 of 128 with a purity of at least 0.9).
2. Repeated runs mattered: at this level of accuracy, a difference of a few hundredths of a percentage point can come from a handful of images or from the random seed, so I needed several seeds to tell real improvements apart from luck.

8.2 Suggested Improvements

Table 7 goes through the improvements suggested in the assignment template plus a few of my own: which ones I already used, which I tested in separate experiments, and possible next steps. Two of them are examined below.

Table 7. Possible improvements and their status in this project.

Improvement	Status	Result / expected effect
Batch normalization	Used	Stable training, even at a learning rate of 10^{-2} (Section 7).
Data augmentation	Used	Prevents memorization (Section 6.1); next: elastic distortion, random erasing.
Deeper architecture (third block)	Used	Remaining errors are mostly ambiguous digits (Section 6.3).
Learning-rate warm-up	Suggested	Could remove the loss spike at epoch 16 (Figure 3).
Ensemble of models	Tested	41→30 errors and better calibration, at 5–25× the cost (Table 8).
Hyperparameter search	Tested	Batch size and stopping rule matter most; tuned model in Table 9.
Fair baseline comparison	Suggested	Same augmentation for both models, to isolate the architecture.
Efficiency	Suggested	<code>torch.compile</code> , bf16, INT8 quantization or pruning.

Experiment 1: ensembling and test-time augmentation (Table 8). I averaged the softmax probabilities of five DigitNets trained with different seeds. For test-time augmentation (TTA), I also averaged each model’s predictions over the original image and four one-pixel shifts. The five individual models averaged 99.588 % (41.2 errors per run). The five-model ensemble made 32 errors (99.68 %), and adding TTA reduced that to 30 (99.70 %), which matched the best single seed rather than beating it. The more noticeable gain was in the probability metrics: compared with the average single model, the ensemble with TTA reduced the log-loss from 0.0120 to 0.0082 and the expected calibration error from 0.0020 to 0.0010. These gains cost 25 forward passes per image.

Table 8. Ensembling and test-time augmentation (TTA) on the test set.

Method	Acc. (%)	Errors	Macro F1	Log-loss	ECE	Passes/img
single model (mean of 5 seeds)	99.588	41.2	0.9959	0.0120	0.0020	1
single model, seed 0 (reported model)	99.70	30.0	0.9970	0.0108	0.0018	1
seed 0 + TTA (5 shifts)	99.67	33.0	0.9967	0.0095	0.0014	5
ensemble of 5	99.68	32.0	0.9968	0.0085	0.0015	5
ensemble of 5 + TTA	99.70	30.0	0.9970	0.0082	0.0010	25

Experiment 2: a tuned training setup (Table 9). Based on the *validation* results of the hyperparameter study, I made two changes:

- **Batch size 32**, which had the lowest validation loss of all settings.
- **Training for all 50 epochs** with the plateau scheduler and keeping the last weights rather than restoring the minimum-loss checkpoint. In the three complete 50-epoch runs, the last-epoch validation accuracy (99.58–99.60 %) was higher than at the epoch with the lowest validation loss (99.50–99.56 %).

Across three new seeds, the tuned models each made 30 test errors ($99.70 \pm 0.00\%$). That is about 0.07 points above the three-seed mean of the default setup (99.63 %), but no better than the default seed-0 run (McNemar $p = 1.000$). The clearer gain is consistency: all three tuned runs reached the same accuracy. Averaging the three tuned models reduced the error count to 23 (99.77%), the highest accuracy in this project. The batch-32 runs from the hyperparameter study, which did use early stopping, scored $99.75 \pm 0.03\%$, so with small batches the stopping rule matters less. The price is compute: about 9 minutes per model instead of about 1.9. Because some of these ideas came from a study that also reported test metrics, I treat this result as an exploratory comparison rather than a fresh, independently held-out benchmark.

Table 9. Tuned setup (batch size 32, 50 epochs with the plateau scheduler, final weights), 3 seeds.

Model	Val. acc. (%)	Test acc. (%)	Errors	Macro F1	Log-loss
tuned DigitNet, seed 0	99.54	99.70	30	0.9970	0.0107
tuned DigitNet, seed 1	99.62	99.70	30	0.9970	0.0108
tuned DigitNet, seed 2	99.64	99.70	30	0.9970	0.0096
tuned ensemble of 3	—	99.77	23	0.9977	0.0082

9. Appendix — Source Code

- **GitHub repository:** <https://github.com/EBARV/CYSE635-MNIST-CNN>. It contains the project notebook `Project-Main file.ipynb`, which holds all of the source code (data pipeline, models, training loop and metrics) and every experiment, with its tables and figures shown as outputs.
- **Key code snippets.** The model definition is shown in Listing 4 below. The residual SE block it uses (Listing 1), early stopping (Listing 2) and the training loop (Listing 3) are in the main text.
- **Graphs and visualizations.** All graphs are included with captions in the main text (Figures 1–11).

Listing 4. Model definition of DigitNet (from the project notebook).

```

1 class DigitNet(nn.Module):
2     """Main model: a compact residual CNN with squeeze-and-excitation attention."""
3
4     def __init__(self, dropout=0.3, widths=(32, 64, 128), hidden=128, n_classes=10):
5         super().__init__()
6         w1, w2, w3 = widths
7         self.standardize = Standardize() # (x - 0.1307) / 0.3081
8         self.stem = nn.Sequential(nn.Conv2d(1, w1, 5, padding=2, bias=False), # conv layer 1: 5x5
9                                   nn.BatchNorm2d(w1), nn.ReLU(inplace=True))
10        self.block1 = ResidualSEBlock(w1, w1) # 32 ch, 28x28
11        self.pool1 = nn.MaxPool2d(2) # 28 -> 14
12        self.block2 = ResidualSEBlock(w1, w2) # 64 ch, 14x14
13        self.pool2 = nn.MaxPool2d(2) # 14 -> 7
14        self.block3 = ResidualSEBlock(w2, w3) # 128 ch, 7x7
15        self.gap = nn.AdaptiveAvgPool2d(1) # global average pooling -> 128 features
16        self.head = nn.Sequential( # classifier: 2 fully connected layers
17            nn.Flatten(), nn.Dropout(dropout),
18            nn.Linear(w3, hidden), nn.ReLU(inplace=True), nn.Dropout(dropout),
19            nn.Linear(hidden, n_classes)) # 10 logits (softmax is in the loss)
20
21        def forward(self, x, return_maps=False):
22            maps = {} # stage outputs, kept for the feature-map plots
23            x = self.standardize(x)
24            x = maps["stem"] = self.stem(x)
25            x = maps["block1"] = self.block1(x)
26            x = maps["block2"] = self.block2(self.pool1(x))
27            x = maps["block3"] = self.block3(self.pool2(x))
28            logits = self.head(self.gap(x))
29            return (logits, maps) if return_maps else logits

```